

Move and remove: Multi-task learning for building simplification in vector maps with a graph convolutional neural network

Zhiyong Zhou^{*}, Cheng Fu, Robert Weibel

Department of Geography, University of Zurich, Zurich, Switzerland

ARTICLE INFO

Keywords:

Map generalization
Building simplification
Vector maps
Multi-task learning
Graph convolutional neural networks

ABSTRACT

Simplification of building footprints is an essential task in topographic map generalization from large to medium scales. The traditional rule- or constraint-based algorithms commonly require cartographers to enumerate and formalize as many scenarios as possible. Recently, some studies have introduced deep learning to image map generalization, whose outputs, however, may exhibit deformed boundaries due to pure image input. Vector maps are thus a reasonable solution to avoid such issues because of their accurate, object-based geometric representation. However, few existing studies have aimed to simplify buildings in vector maps with the help of neural networks. Building simplification in vector maps can be regarded as the joint contribution from two elementary operations on vertices of building polygons: remove redundant vertices and move kept vertices. This research proposes a multi-task learning method with graph convolutional neural networks. The proposed method formulates the building simplification problem as a joint task of node removal classification and node movement regression. A multi-task graph convolutional neural network model (MT_GCNN) is developed to learn node removal and movement simultaneously. The model was evaluated with a map from Stuttgart, Germany that contains 8494 buildings generalized from the source scale of 1:5,000 to the target scale of 1:10,000. The experimental results show that the proposed method can generate 80% of the buildings with positional errors of less than 0.2 m, 95% with a shape difference under 0.5, and around 98% with an area difference under 0.1 of IoU, compared to the ground truth target buildings, thus demonstrating the feasibility of the proposed method. The code is available at: <https://github.com/chouisgiser/MapGeneralizer>.

1. Introduction

In map generalization, buildings are one of the main object classes in topographic maps (Brassel and Weibel, 1988; Regnauld and McMaster, 2007; Mackaness et al., 2014). Most topographic databases today are captured at very large scales of 1:1000 to 1:5000, while most topographic maps are displayed at 1:10,000 and smaller. In the transition from the high-resolution topographic database to topographic maps, building footprints have to be simplified. Building simplification is the dominating generalization operator in topographic maps of large to medium scales (i.e. up to about 1:25,000), while contextual generalization operators, such as aggregation, typification or displacement, only become more important at medium to smaller scales (Regnauld and McMaster, 2007; Harrie and Weibel, 2007). For the purpose of building simplification, various algorithms have been proposed by considering the geometric characteristics of building shapes, such as edge length (Regnauld et al., 1999; Sester and Brenner, 2005), edge number (Haunert and Wolff, 2010; Samsonov and Yakimova, 2017), and building area (Buchin et al., 2016). Recently, some scholars proposed

to use machine classification to select the most appropriate one from a set of algorithms for simplifying buildings of different shapes (Yang et al., 2022).

The rapid progress of deep learning techniques in other fields also holds the potential of advancing automated map generalization from the algorithm-based to the data-driven era (Touya et al., 2019). The majority of deep learning solutions for map generalization focus on image maps and formulate the map generalization problem akin to the semantic segmentation task in the field of computer vision (Feng et al., 2019; Courtial et al., 2023). While these image-based solutions have shown rather promising results in other applications, they invariably impose additional noise in their representation of building edges and are prone to cause deformed building footprints due to the input maps consisting of pure images.

Therefore, it is reasonable to directly exploit vector maps to simplify building shapes to guarantee visually coherent building boundaries, and it is also an important rationale for most existing algorithm- and rule-based building simplification approaches to mainly focus on the

^{*} Corresponding author.

E-mail address: zhiyong.zhou@geo.uzh.ch (Z. Zhou).

<https://doi.org/10.1016/j.isprsjprs.2023.06.004>

Received 6 January 2023; Received in revised form 8 June 2023; Accepted 12 June 2023

Available online 21 June 2023

0924-2716/© 2023 The Author(s). Published by Elsevier B.V. on behalf of International Society for Photogrammetry and Remote Sensing, Inc. (ISPRS). This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

vector maps (Sester and Brenner, 2005; Haunert and Wolff, 2010; Samsonov and Yakimova, 2017; Buchin et al., 2016; Yang et al., 2022). A few studies have attempted to apply graph convolutional neural networks (GCNNs) to recognize building structures (Yan et al., 2019), encode building shapes (Yan et al., 2021), group (Yan et al., 2022), and select buildings (Xiao et al., 2023). However, they mainly take place at the pre-processing stage of map generalization, while the actual building simplification of vector maps, which belongs to the formal stage of map generalization, was never studied with GCNN, to the best of our knowledge. It is thus currently still unclear to what extent GCNNs can enable automated building simplification.

To explore the capability of deep learning techniques, particularly graph convolutional neural networks, this research proposes a multi-task learning method with GCNN. The polygonal boundaries of buildings are encoded with polygon graphs where each node represents a vertex on the building footprint and an edge denotes a part of the boundary of the building. Inspired by the fact that the simplification of buildings can be achieved by elementary vertex operations, including removing and moving vertices (Sester, 2005), the building simplification problem in this research is formulated as a joint task of node removal classification and node movement regression, respectively. Consequently, a multi-task graph convolutional neural network (MT_GCNN) model is developed to simultaneously learn the node removal and node movement knowledge with graph convolutions. The first version of the implementation code is released at: <https://github.com/chouisgiser/MapGeneralizer>. The main contributions of this research are as follows:

- (1) The building simplification of vector maps is formulated as node removal classification and node movement regression that are learnable for GCNN architectures. A series of experiments with around 1700 sample buildings demonstrate that the solution is feasible to generate reasonable simplifications for most buildings.
- (2) For the first time, the multi-task learning method is introduced to deep learning-based map generalization, which allows for the flexible extension of multiple generalization constraints. The experimental results show that compared with single-task learning, multi-task learning improves the node movement regression while having no negative impacts on the node removal classification.
- (3) The preparation of a training dataset that both considers input features and the control of building complexity is empirically highlighted as a crucial foundation for the success of deep learning in building simplification.
- (4) One particular GCNN architecture, GraphSAGE (SAmple and aggreGatE, Hamilton et al., 2017), is identified as a more effective and efficient graph convolutional operation in our MT_GCNN model compared with the other two representative operations used, graph convolutional network (GCN) and graph attention network (GAT).

2. Related work

2.1. Traditional building simplification algorithms

Traditional building or polygon simplification algorithms mainly operate on vertices, edges, or areas that consist of polygonal shapes (Regnauld and McMaster, 2007). To simplify those polygons that have excessively short edges and small extrusions, vertex-removal operators are one of the popular options based on the classical Douglas–Peucker algorithm (Douglas and Peucker, 1973), or algorithms using convexity (Wang et al., 2005), or orientation (Bayer, 2009) criteria. Sester and Brenner (2005) proposed a set of meta operators on individual polygon vertices, including remove, move, duplicate, and insert vertex, to simplify building polygons. With a view on preserving the shape

characteristics of buildings, simplification algorithms have been proposed that focus on edges, including removal and extension of edges based on edge length (Regnauld et al., 1999; Sester, 2005), edge numbers (Haunert and Wolff, 2010), or edge convexity (Buchin et al., 2016). Besides, some area-based approaches have been considered to simplify polygons using hierarchical decomposition (Ai et al., 2005), template matching (Yan et al., 2017), superpixel model (Shen et al., 2019), fractal theory (Ma et al., 2020), and morphological operators (Wang et al., 2023). Recently, Yang et al. (2022) employed the ensemble mechanism and trained a classifier to choose the most appropriate one from a set of traditional algorithms for the simplification of buildings of different shapes.

Furthermore, to ensure the legibility of simplified maps, a lot of constraints acting on individual buildings and between buildings, such as transformation distance (Sester, 2005) or composite edge number (Haunert and Wolff, 2010), have been imposed to find optimal simplification solutions through optimization techniques. Besides, the group patterns of buildings have also been introduced to decide on the degree of simplification of buildings based on certain criteria, such as symmetry (Haunert, 2012), character shapes (He et al., 2018), and shape regularity (Yan et al., 2019). While these traditional algorithms introduce intuitive approaches to building simplification, their main disadvantage lies in the manual setting of threshold parameters by trained cartographers, which limits their widespread use by less experienced users. They also necessitate formalizing the implicit knowledge that expert cartographers employ, and that can be harnessed as constraints and rules forming the basis of appropriate cartographic generalization algorithms. This need for making implicit cartographic knowledge explicit before any adequate algorithmic solution can be developed has also been termed the knowledge acquisition bottleneck (Weibel et al., 1995) and has severely hampered the development of full-scale solutions in map generalization for decades.

2.2. Deep learning in building simplification

The advance of deep learning techniques has been revolutionizing a wide range of fields, including cartography and geographic information science, with their strong capacity for representation learning of data. In map generalization, existing deep learning approaches mainly convert original vector spatial data to raster images, following which the generalization is formulated as a more straightforward semantic segmentation problem for deep learning models. Besides the early back-propagation neural networks (Cheng et al., 2013), the more advanced deep learning neural networks including U-Net (Feng et al., 2019), generative adversarial networks (GAN) (Courtial et al., 2023), and their variants such as Residual U-Net (Feng et al., 2019) were applied.

Unfortunately, the inherent issue of image maps is that they are imposing additional noise on maps, which is prone to cause deformed boundaries of buildings polygons in maps (Feng et al., 2019). Therefore, it is reasonable to adopt the original vector map as the input for deep learning and predict the output in vector format as well. Graph neural networks have shown a promising capacity for encoding vector data (Mai et al., 2022a) and relevant tasks related to map generalization. For instance, Yan et al. (2019) formulated recognizing regularity of different blocks of buildings in urban environments as a graph-level classification problem and addressed it by introducing a graph convolutional neural network where each node represents an individual building and each edge represents the spatial proximity between a pair of buildings. Likewise, they further applied such a model to the building grouping task for map generalization (Yan et al., 2022). Besides learning the spatial layout among buildings, graph convolutional neural networks were also used to encode the shapes of individual buildings by representing a vertex on the polygonal boundary of a building as a node and the connectivity between vertices as an edge (Yan et al., 2021). However, these studies solely focus on the pattern and structure recognition step (Brassel and Weibel, 1988) preceding the

generalization process, while research on the actual *simplification* of buildings in vector maps is still lacking. Zhou et al. (2022) proposed to use GCNN to deal with the building simplification of vector maps, but their work is solely staying in the conceptual stage.

Compared to the traditional rule- or constraint-based methods, where generalization operators are triggered by the rules or constraints customized by expert cartographers, with the challenge that different rules may be in conflict with each other, the data-driven deep learning approaches are freed from explicitly formalizing potentially conflicting rules and find the most appropriate global solution for map generalization solely by learning from examples. Even the most sophisticated constraint-based systems, represented by the agent-based approach (Barraut et al., 2001; Ruas and Duchêne, 2007)—while capable of adapting to given map generalization situations and of choosing their path of operation depending on the constraints that are violated and to the degree that these constraints are violated—are not capable of learning and, importantly, are not capable of learning from examples, as a DL model does.

3. Methodology

3.1. Graph representation of buildings

Given the great capacity of graphs to represent shape characteristics of building polygons (Yan et al., 2021), this research follows a similar approach to representing polygons of individual buildings of vector maps. For the understanding, we have the following definitions in which the *polygon graph* is a term for the graph representation of buildings, and the other two definitions *feature matrix* and *adjacency matrix* formalize the mathematical input of the graph representation for the proposed method.

Definition 3.1 (Polygon Graph). An undirected cycle graph corresponding to the polygon of an individual building. In the graph, each node¹ represents a vertex of the building polygon and each edge refers to a part of the building polygon connecting two vertices. Fig. 1 illustrates an example of a polygon graph. Notably, different from Yan et al. (2021), where the boundary geometry of a building polygon is segmented by a certain number of vertices (e.g., 64) to unify the input size of graphs for deep learning, our approach keeps the original vertices that define the footprint of a building polygon. Such a form has a more compact and compatible representation of building polygons with different shapes and areas.

Definition 3.2 (Feature Matrix). An $N \times D$ matrix, where N is the number of nodes that construct a polygon graph and D is the number of characterizing features per node. According to the feature engineering as elaborated in Section 4.4.1, this research uses four features to characterize each node of a polygon graph (Yan et al., 2021; Du et al., 2022), including the length of the preceding edge and the succeeding edge, denoted by $Len_1(v_i)$ and $Len_2(v_i)$, and the rotation angle $AgI(v_i)$ and direction $Dir(v_i)$ from the preceding edge to the succeeding edge for a given node v_i as illustrated in Fig. 1.

Definition 3.3 (Adjacency Matrix). An $N \times N$ matrix representing the adjacency between each pair of nodes in a polygon graph. If there is an edge connecting a pair of nodes, the adjacency is denoted as 1, otherwise, the adjacency is 0.

¹ We use the term *node* in this paper. In graph theory the term *vertex* is also commonly used as a synonym.

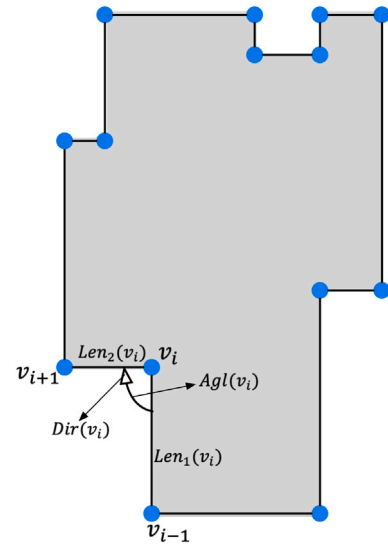


Fig. 1. Graphical illustration of a polygon graph and the calculation of its node features (Yan et al., 2021; Du et al., 2022): Given a current node v_i , its preceding node v_{i-1} , and succeeding node v_{i+1} , $Len_1(v_i)$ and $Len_2(v_i)$ represent the length of the preceding edge and the succeeding edge of v_i , respectively; $AgI(v_i)$ is the rotation angle from the preceding edge $\overline{v_{i-1}v_i}$ to the succeeding edge $\overline{v_iv_{i+1}}$ and $Dir(v_i)$ is a signed float representing the rotation direction, i.e., positive for clockwise and negative for counter-clockwise, respectively.

3.2. Problem definition

Inspired by the fact that building simplification can be achieved by elementary operations on vertices, including *remove*, *move*, *duplicate*, and *insert vertex* (Sester and Brenner, 2005), this research formulates the simplification of buildings as a joint task of two sub-tasks as shown in Fig. 2: removing and moving vertices of individual building polygons. Note that the other two elementary operators, i.e., *duplicate* and *insert vertex*, are not included in the joint task because the polygons of buildings to be simplified commonly have redundant vertices compared to the generalized polygon and thus it is unnecessary to introduce extra vertices through these two meta operators. Before formally defining the problem of building simplification, two tasks are first introduced below. In the context of a polygon graph, the vertex of a polygon is termed a node.

Definition 3.4 (Node Removal Classification). As shown in the upper part of Fig. 2, one task of building simplification is to decide for each node of a source polygon graph if it should be *removed*, *moved*, or *kept static* to construct the target polygon graph in the generalized map. The three decided states of nodes can be labeled as 0, 1, and 2, and such a process is thus modeled as a *node removal classification* task.

Definition 3.5 (Node Movement Regression). The lower part of Fig. 2 illustrates the other task of building simplification, that is, to calculate a numerical displacement vector for those polygon nodes in the source map that have been classified to be *moved* in the *node removal classification* task. This part is formulated as a *node movement regression* task. In this research, the displacement vector is composed of the displacement distance of a node at a local reference frame formed by two non-collinear edges, that is, the directions of its preceding edge and succeeding edge, respectively (see the red dashed lines in Fig. 2). The distance is a signed numerical value, where positive values refer to an edge to be extended along the edge direction, while negative values refer to an edge to be shortened along the edge direction.

Overall, the graph-based building simplification problem is defined as follows:

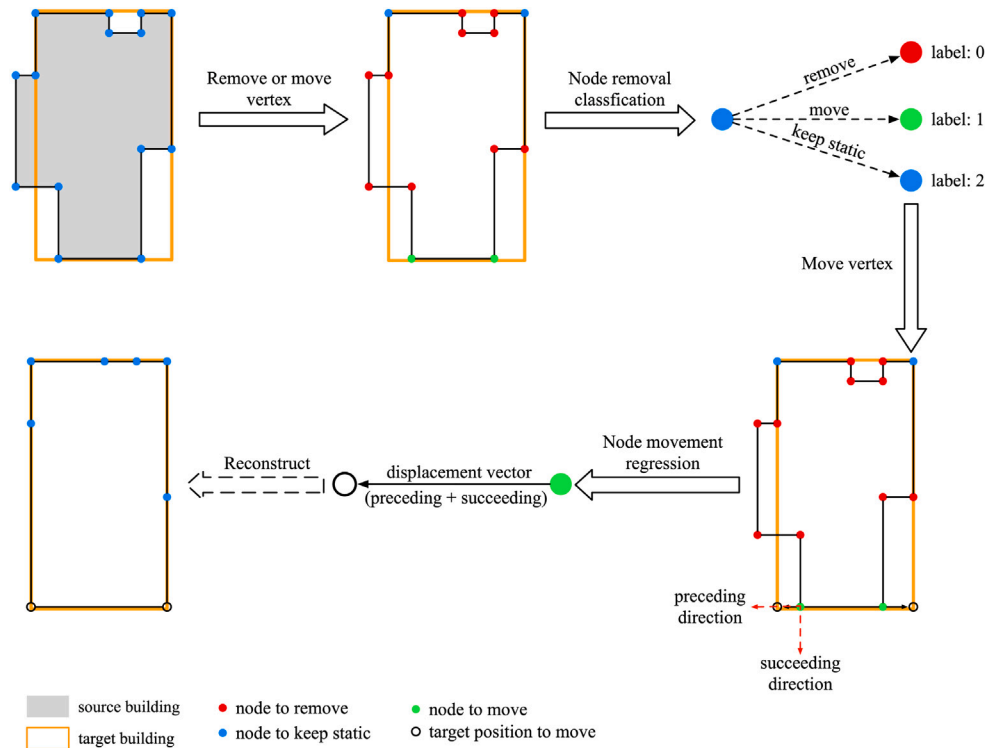


Fig. 2. Building simplification as a joint task with node removal and node movement. The node removal classification is first conducted to judge for each node of a polygon graph whether it should be removed (0), moved (1), or kept static (2), after which the node movement regression will be performed on the moved nodes.

Problem 1 (Building Simplification). Given a polygon graph G from the source map and its feature matrix X , the problem of building simplification is to learn the mapping function: $f(G, X) \rightarrow Y^{(0,1,2)}$, where $Y^{(0,1,2)}$ refers to a three-item tuple representing the node operations from the source building to target building, including three-class labels for *node removal classification*, and numerical movement distance along preceding and succeeding edges, respectively, for *node movement regression*.

3.3. MT_GCNN: Multi-task graph convolutional neural network model

3.3.1. Overall model architecture

Targeting multiple outputs with three-class labels for node removal classification and numerical values for node movement regression, a multi-task graph convolutional neural network (MT_GCNN) model is proposed in this research. As the node movement regression depends on the node removal classification results and mainly focuses on those nodes that are classified as to move, as illustrated in Fig. 3, we adopt a hierarchical architecture for the multi-task learning that allows fusing the encoded features from the node removal classification task with the original polygon graph features in the node movement regression task. The key components of the proposed model are introduced below:

- **Input.** As defined in Section 3.1, the polygonal boundaries of individual buildings on vector maps are represented by polygon graphs. The 4-dimensional node feature matrix and adjacency matrix can thus be constructed based on the graphs and then serve as the input of the model.
- **Node removal classification layers.** As shown in red boxes in Fig. 3, three graph convolutional neural (GCN) layers, which sequentially produce 32-dimensional, 16-dimensional, and 3-dimensional features, are introduced to extract polygon graph features for the node removal classification. Eventually, a Log-Softmax activation function is applied to output three-class node removal labels, i.e., 0 (removed), 1 (moved), and 2 (kept static) in Fig. 2.

- **Node movement regression layers.** The node movement regression task is further composed of two sub-tasks, which are to predict the displacement distance of each node both along the preceding and succeeding edges, respectively. The corresponding layers are colored blue in Fig. 3. Initially, since the node displacement should be mainly applied to those nodes that are classified to move from the node removal classification layers, the input node features for the movement regression are extended to 7 dimensions by concatenating the extracted 3-dimensional features from the node removal classification layers with the original input 4-dimensional features. To enable to share the similar features to regress displacement distance, a shared GCN layer (see the dark blue box in Fig. 3) is first introduced, after which two layers of graph convolutions are applied to encode graph features for each movement along two directions of the node. Finally, a fully connected (FC) layer is used to output the displacement vector.
- **Regularization layers and activation function.** It should be noted that a batch normalization layer and a ReLU activation function are placed between every two GCN layers. The former normalizes each batch of input graphs and serves as a regularization function to guarantee the generalization ability of the model, while the latter mainly deepens the modeling of non-linear relationships between these hidden layers.
- **Output.** The model outputs a 3-dimensional vector as defined in Problem 1, the first of which corresponds to a three-class label to indicate if a node should be removed (0), moved (1), or kept static (2) to construct the polygon of a building in the target map. The second and third dimensions represent the displacement distance of nodes that are predicted to move in the node removal classification along the preceding and succeeding edges, respectively. Considering that the displacement distance of those nodes predicted to be removed and kept static is 0, the FC layer outputs in the node regression are multiplied by the node removal classification results to yield the final displacement distance for nodes.

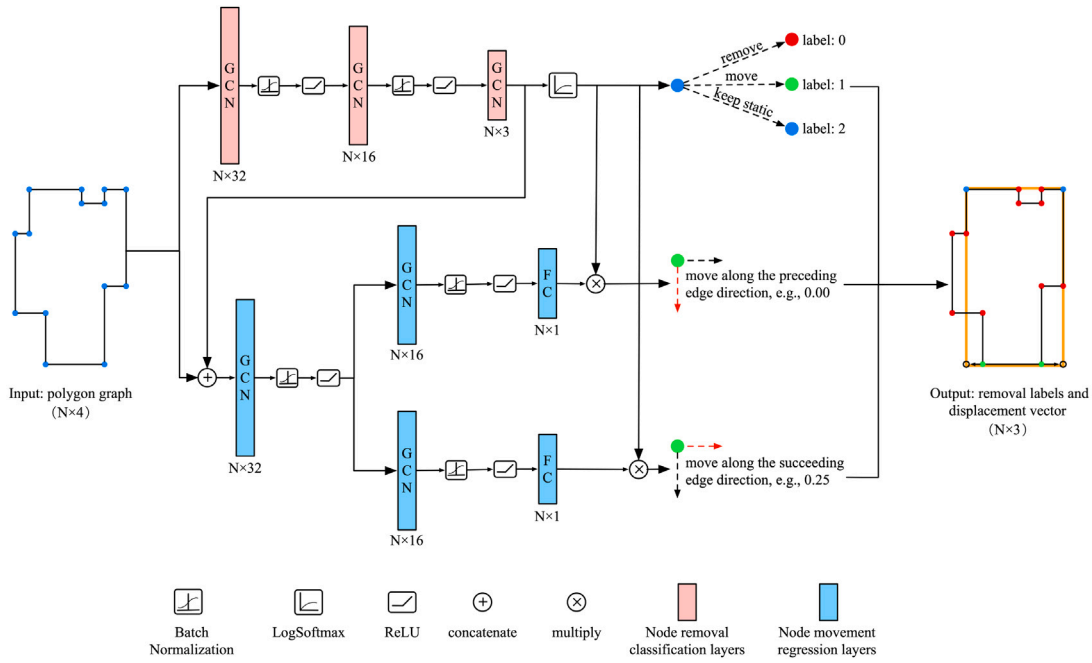


Fig. 3. The architecture of MT_GCNN: Multi-task graph convolutional neural network model. It is composed of two modules: the node removal module (pink boxes) and the node movement regression module (blue). The latter module can then be further separated into two sub-tasks, i.e., move along the preceding edge direction (see the middle row of boxes) and move along the succeeding edge direction (see the bottom row of boxes). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3.3.2. GraphSAGE convolution operation

There are a number of graph convolution operators, such as the conventional graph convolution operator (GCN) (Kipf and Welling, 2017), graph attentional operator (GAT) (Velickovic et al., 2018), and GraphSAGE operator (Hamilton et al., 2017). As the layer propagation in a convolutional neural network does, the node embeddings at a certain layer of the GCN are updated by aggregating its previous layer's node embeddings based on the adjacency matrix and the degree matrix of the whole graph. Considering that the aggregation of node embeddings in a GCN does not allow for the importance of an edge connecting nodes, GAT is thus developed to aggregate node features with an attention weight. However, the update of node features in GCN and GAT both depends on the whole graph's adjacency matrix and degree matrix, which means the structure of the whole graph (e.g., number of nodes) closely relates to the generation of node embeddings, and GCN and GAT are thus not flexible to learn node embeddings of graphs with an unfixed number of nodes. Besides, such a global update also negatively affects the computational efficiency of learning.

GraphSAGE has the capacity to address these limitations. Its essential difference from GCN and GAT is that its objective is to learn a set of aggregator functions to induce and update node embeddings locally by sampling a certain number of neighboring nodes instead of updating node embeddings based on the whole graph structure as in GCN and GAT. Thus, GraphSAGE has a better generality to learn embeddings for graphs with unfixed nodes and is also more computationally efficient than GCN and GAT. As each polygon graph for individual buildings has varying numbers of nodes, which means the structures of polygon graphs differ from each other, GraphSAGE is thus selected to perform graph convolution operations involved in the proposed model as shown in Fig. 3. Notably, the sampling of neighboring nodes in GraphSAGE is randomly performed to get the given number of nodes. However, as our conceptualized polygon graph is a cycled graph where each node always has two neighboring nodes, which is a small size, we directly set the sampling size to 2 to make the model remove the sampling effect.

Fig. 4 illustrates the process of generating an embedding of a node in a polygon graph, h_v^{i+1} , at layer $i+1$ based on the previous layer i , assuming that the embeddings of a node v and its neighboring nodes

$n1, n2$ are h_v^i, h_{n1}^i , and h_{n2}^i at layer i , respectively. The GraphSAGE convolutional layer samples the neighboring nodes of a given node v , which are $n1$ and $n2$,² after which the embeddings of neighboring nodes at layer i , h_{n1}^i and h_{n2}^i , are aggregated to result in h_n^i based on certain criteria, such as mean, max, and LSTM (Hamilton et al., 2017). In this research, we adopt the most commonly used min aggregator to aggregate embeddings of neighboring nodes. Then, the embedding of the given node v itself at layer $i+1$, h_v^i , is concatenated with the aggregated neighboring embedding h_n^i to denote the embedding of node v at layer $i+1$ with certain W_{i+1} that should be learned through data feeding.

3.3.3. Loss function for multi-task learning

As defined in Section 3.2, a classification task for node removal and a regression task for node movement jointly contribute to the building simplification of vector maps. As the node movement regression is subject to the node removal classification that decides whether nodes move or not, it is reasonable to regard the node removal classification as an auxiliary task for the node movement regression. In this research, we introduce a total loss function that combines the node classification task and the node movement task considering the auxiliary relationships between tasks (Liebel and Körner, 2018; Liu et al., 2021) as defined below:

$$L_{bldgs_sim} = \frac{1}{w_1^2} \cdot L_{rm} + \frac{1}{w_2^2} \cdot L_{pre_mv} + \frac{1}{w_3^2} \cdot L_{suc_mv} + \sum_{i=1}^3 \log(w_i + 1) \quad (1)$$

where L_{bldgs_sim} is the total loss for the building simplification, L_{rm} refers to the node removal classification loss, L_{pre_mv} refers to the loss of node movement along the preceding edge, and L_{suc_mv} refers to the loss of node movement along the succeeding edge. Meanwhile, the loss of each task is weighted by w_i , $i \in \{1, 2, 3\}$, respectively. Notably, the weights are learned from data feeding and thus free of manual settings.

² It should be noted that the original sampling of GraphSAGE should set a certain sample size. However, since the polygon graph is a cycle graph, the sample size is constantly set as 2.

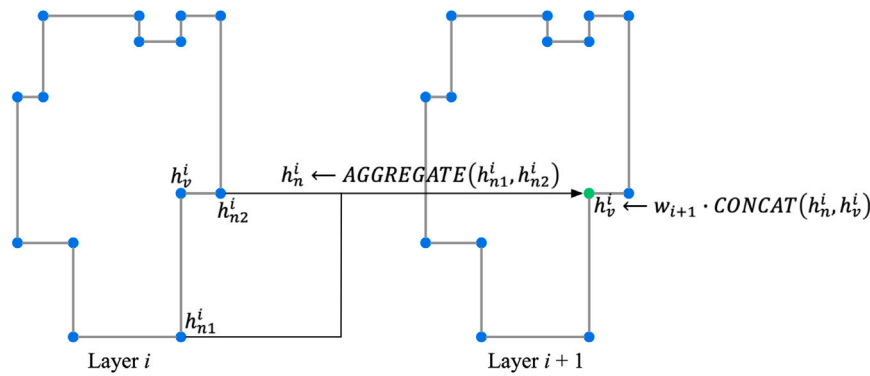


Fig. 4. The process to generate the polygon graph node embedding using GraphSAGE.

The detailed learning process is discussed in [Liebel and Körner \(2018\)](#). Specifically, we use the negative log-likelihood as the loss function for L_{rm} and the mean absolute error loss for L_{pre_mv} and L_{suc_mv} .

4. Experiment and evaluation

To evaluate the performance of the proposed model on building simplification of vector maps, a set of experiments were conducted following the questions identified below:

- (1) How advantageous is the multi-task learning strategy over the single-task learning paradigm to learn the building simplification knowledge? (Section 4.2)
- (2) How feasible is the proposed method to simplify buildings of vector maps? (Section 4.3)
- (3) How effective is the parameter setting in training the proposed model? (Section 4.4)

4.1. Experimental setup

4.1.1. Dataset preparation

We used the Stuttgart building data from [Feng et al. \(2019\)](#) as the basis for training the proposed MT_GCNN model. Specifically, the map generalization products resulted from a software called CHANGE, which provides the generalization module used to simplify and amalgamate buildings with certain building facade width and area thresholds as well as the distance thresholds between buildings ([Powitz, 1993](#); [Bildirci, 2004](#)). In this research, the scale of 1:5000 was selected as the source map, and the scale of 1:10,000 was used as the target map. Several processes were imposed on the vector-format building data of the Stuttgart dataset, which initially contains 117 254 buildings. Firstly, we excluded the buildings that have wrong joint ID correspondence between the source map and the target map and were generated by the amalgamation operator. Considering that a very small number of buildings (e.g., churches) with extremely complicated shapes may negatively influence the model to learn the more common cartographic knowledge, we also filtered out the top 1% of buildings regarding the number of vertices contained in their footprints (minimum: 4; maximum: 19 in the remaining buildings). Besides, since there are usually a large number of buildings not being simplified from the source to the target map (e.g., because their footprint is a simple rectangle), far more than the simplified buildings, some of the non-simplified buildings were randomly excluded to make their number equivalent to the number of simplified buildings. Consequently, the final dataset consisted of 8494 buildings, half of which have simplified shapes in the target map, while the other half are not simplified.

As explained in Section 3.3.1, besides the input 4-dimensional features for each node of a polygon graph, the corresponding output 3-dimensional vector should be labeled for training the model. For the three-class label of node removal, we first aligned the source polygon

graph and the target polygon graph through the iterative closest point (ICP) registration algorithm,³ after which those nodes in the source polygon graph that failed to be mapped to a node in the target polygon graph were labeled as 0, that is, *removed*. Afterward, the distance between each pair of mapped nodes was calculated. Those pairs of nodes whose distance is smaller than 0.1 m were labeled as 2, that is, *kept static*, while the remaining nodes were labeled as 1, meaning to be *moved*.

In terms of node movement, the displacement distance of those nodes with the node removal label of 0 or 1 is directly assigned with the value of 0.0 for its preceding and succeeding edge, respectively. Then, for nodes to be moved, taking each node as the origin and its preceding and succeeding edges as two axes to construct a local reference frame, the target position was then projected to two axes. The distance from the origin to the projected point was taken as the node displacement distance along one direction. For example, in [Fig. 5\(a\)](#), the source node s and its preceding and succeeding edge directions⁴ constitute a local reference frame. The target node t is then projected to the axes of the preceding and succeeding directions, respectively. In this example, the projected distance from the origin (i.e., the position of s) is 0.25 along the preceding direction and 0.00 along the succeeding direction. As a result, the movement vector is coded as (0.25, 0). It should be noted that the local reference frame is not certainly orthogonal because the boundary shape of a building can be arbitrary. As shown in [Fig. 5\(b\)](#), however, it is still feasible to calculate a displacement vector through the projection to the non-collinear preceding and succeeding edges.

4.1.2. Dataset description

The built dataset was split with a ratio of 6:2:2, that is, 5096, 1700, and 1698 buildings, for the training, validation, and testing. We introduced the number of vertices per building polygon and the shape deviation from a rectangle (SDR)⁵ to capture the complexity of the simplification task for each building in the dataset ([Basaraner and Cetinkaya, 2017](#)). As shown in [Fig. 6](#), the SDRs of building footprints at both the training stage and the testing stage vary mainly from 0.0 to 0.3 (with 0.0 denoting a rectangular building footprint), while the number of vertices per building mainly range between 4 (a rectangle) and 30, with only very few values above 30, denoting highly complex footprints, such as historical churches or cathedrals. The distributions indicate that the building shapes in the dataset are not purely regular rectangles and thus hold sufficient shape diversity for training models for the simplification task. Such distributions comply with the shape distribution of building footprints in urban environments.

³ http://www.open3d.org/docs/release/tutorial/pipelines/icp_registration.html.

⁴ Assume the polygon boundary is arranged in clockwise order.

⁵ $SDR = 1 - \frac{\text{the ratio of a building polygon's area to its minimum rotated rectangle's area}}$

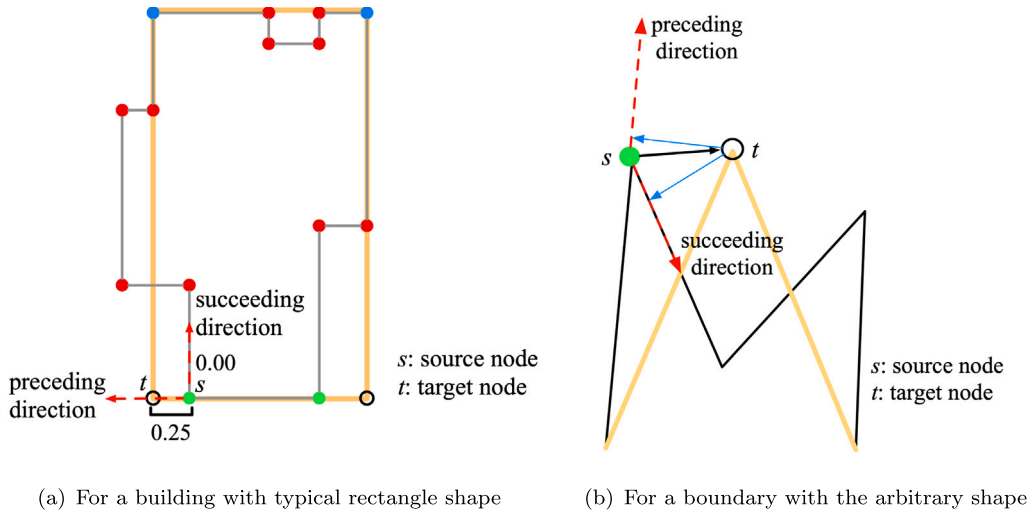


Fig. 5. Examples to calculate the movement vector along the preceding edge direction and the succeeding edge direction.

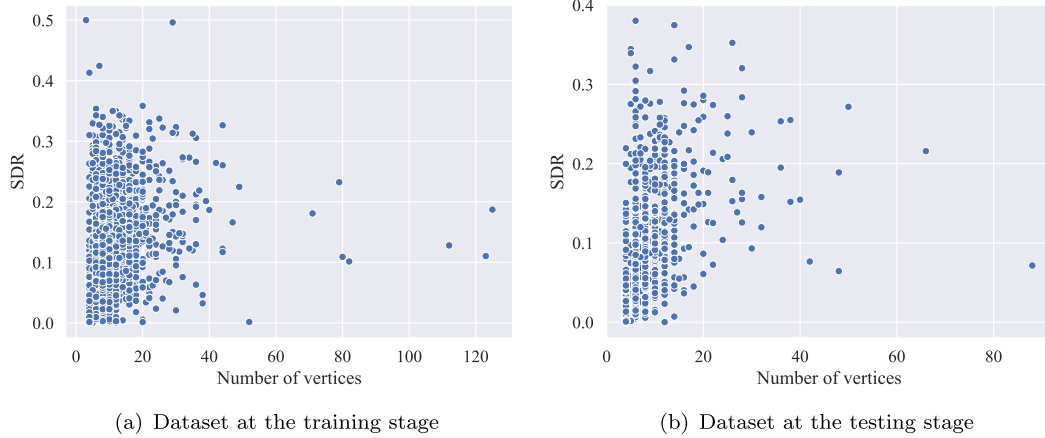


Fig. 6. The distribution of geometric complexity of building polygons contained in the training and testing dataset, respectively.

4.1.3. Model implementation details

We utilized the PyTorch 1.9.0 deep learning framework and PyTorch Geometric graph learning library to implement the proposed MT_GCNN model under the Python 3.7.11 environment. To train the MT_GCNN model, the training epoch number was set as 200 and the batch size was 64. We chose the Adam optimizer (Kingma and Ba, 2015), the learning rate of 5×10^{-4} , and the weight decay of 1×10^{-6} to optimize the weights of features and losses of multiple tasks.

4.1.4. Evaluation metrics

The proposed method was evaluated in two aspects, the internal evaluation of the trained model (Section 4.2) and the external evaluation of the building simplification products that were reconstructed from the predictions of the trained model (Section 4.3). The former includes the accuracy and an F1 score to quantify the node removal classification performance and the mean absolute error (MAE) to measure the node movement regression performance.⁶ Regarding the external evaluation, the polygons of simplified buildings were first reconstructed based on the predicted node removal labels and displacement vectors, after which they were evaluated in terms of positional

error, shape difference, and area difference from the ground truth polygons, that is, the buildings at the target scale. The used metrics for these three aspects are Hausdorff distance (HD), turning function distance (TFD), and intersection over union (IoU), respectively (Yang et al., 2022). The lower the HD and the TFD are and the higher the IoU is, the closer the simplified building by our method is to the target building, indicating better results.

4.2. Internal evaluation of the trained model

The validation accuracy of the node removal classification and the MAE of the node movement regression during the training phase are illustrated in Fig. 7. It shows that the classification accuracy increases significantly from 0.62 to 0.80 after 20 epochs and then tends to stay stable after 200 epochs, although it still fluctuates between 0.80 and 0.85 from epoch to epoch. Meanwhile, the MAE of node movement regression holds a similar trend ranging between 0.20 m and 0.30 m after 200 epochs. Hence, we conclude that the trained model after 500 epochs has the stable generality to evaluate with the test set.

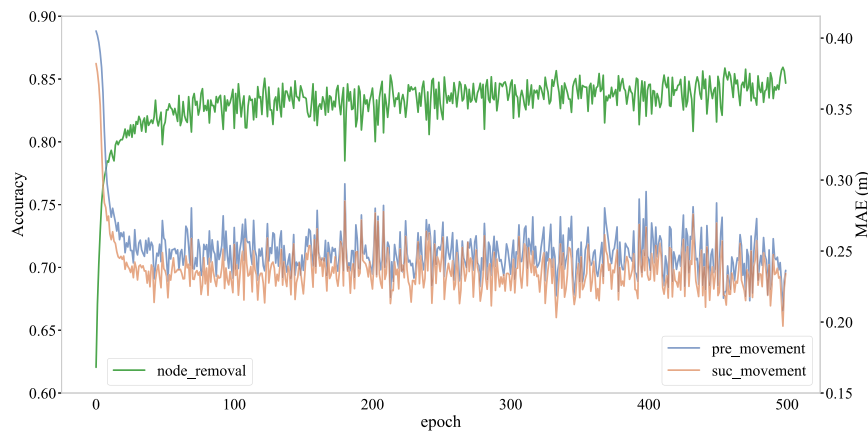
The internal evaluation results of the trained model based on the multi-task learning method are reported in Table 1. The model achieved the node removal classification accuracy of 0.862 and F1 score of 0.926. Regarding the node movement regression results, the model had an MAE of 0.390 m displacement along the preceding edge and an MAE of 0.388 m displacement along the succeeding edge. Besides, as shown in Table 1, the model is also compared to the three models that are

⁶ Since there are a large number of buildings that have not been simplified from the source map to the target map, which diminishes the error of the actual simplification, we excluded these buildings with the movement distance threshold of 0.01 m and only calculated the MAE of the generalized buildings.

Table 1

Comparing results of the proposed multi-task learning method with single-task learning methods.

Learning paradigms		Node removal		Node movement - preceding	Node movement - succeeding
		Accuracy	F1	MAE	MAE
Multi-task	Node removal + movement (ours)	0.862	0.926	0.390	0.388
Single-task	Node removal only	0.866	0.928	–	–
	Node movement (preceding) only	–	–	0.428	–
	Node movement (succeeding) only	–	–	–	0.424

**Fig. 7.** Validation accuracy of the node removal classification and validation MAE of the node movement regression.

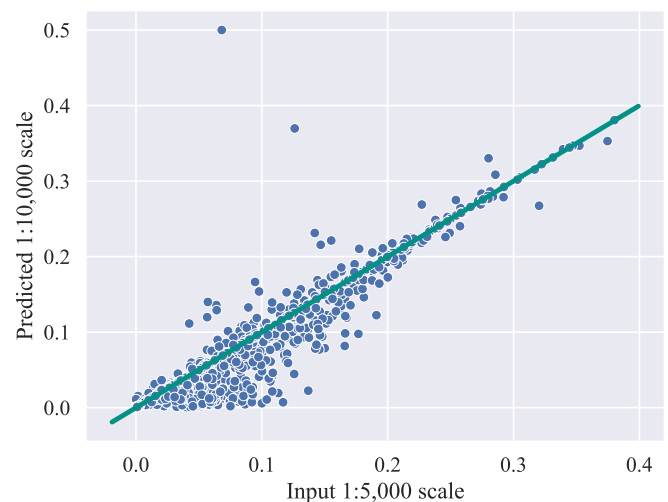
trained with a single-task learning paradigm for node removal only and node displacement along the preceding edge and the succeeding edge, respectively. As a result, the node removal classification performance of our proposed multi-task learning model is slightly lower than that of the single-task learning model (accuracy: 0.866, F1 score: 0.928), while our model has smaller MAEs with 0.038 m and 0.036 m for node regression movement regression along the preceding edge and the succeeding edge, compared with those from the single-task learning model (preceding: 0.428 m, succeeding: 0.424 m). The internal evaluation results suggest that the multi-task learning paradigm does not harm the node removal classification and is beneficial for the node movement regression.

4.3. External evaluation of the building simplification products

The simplified buildings at the target scale, that is, 1:10,000 in this research, were reconstructed based on the predicted node removal labels and node movement vectors for individual nodes of buildings at the source scale. Since half of the used dataset consisted of non-simplified buildings, which mimics the realistic building simplification process where a large number of buildings need not be simplified in the scale transition from 1:5000 to 1:10,000, the global evaluation with all buildings may conceal the performance of the proposed method on those cases where the simplification actually happens. Therefore, besides the global evaluation with all test buildings in Section 4.3.1, we also conducted local evaluations with test buildings at different simplification levels, reported in Section 4.3.2. Specifically, the simplification levels were regulated based on the IoU between the source map at the scale of 1:5000 and the ground truth target map at the scale of 1:10,000, with this IoU giving an indication of the degree of generalization that should be expected to take place.

4.3.1. Global evaluation of all buildings

Fig. 8 illustrates a comparison of the geometric complexity of individual building footprints between the input and the prediction in terms of SDR. It shows that the majority of data points are distributed under the diagonal, indicating that the trained model is capable of simplifying building footprints in spite of some few cases sitting above the diagonal.

**Fig. 8.** The geometric complexity, in terms of SDR, of the individual building footprints between the 1:5000 scale source map and the predicted 1:10,000 scale map.

Furthermore, the Hausdorff distance (HD), turning function distance (TFD), and intersection over union (IoU) were used to evaluate simplified individual building footprints concerning the positional error, shape difference, and area difference compared to the corresponding buildings at the target map. Fig. 9 illustrates the empirical cumulative distributions (eCD) of the evaluation results with the introduced three metrics. It is clear that all three metrics exhibit a heavy-tailed distribution. Fig. 9(a) shows that 80% of the simplified buildings have a positional error, measured by the HD, smaller than 0.5 m and nearly 100% of errors are below around 3.5 m. Regarding the shape difference, which is reflected by the TFD shown in Fig. 9(b), approximately 95% of buildings have a TFD below 0.5, which is recognized as a threshold value to denote high similarity of polygons (Arkin et al., 1991). Besides, the eCD curve in Fig. 9(c) shows that the majority of simplified buildings have an IoU between 0.9 and 1.0.

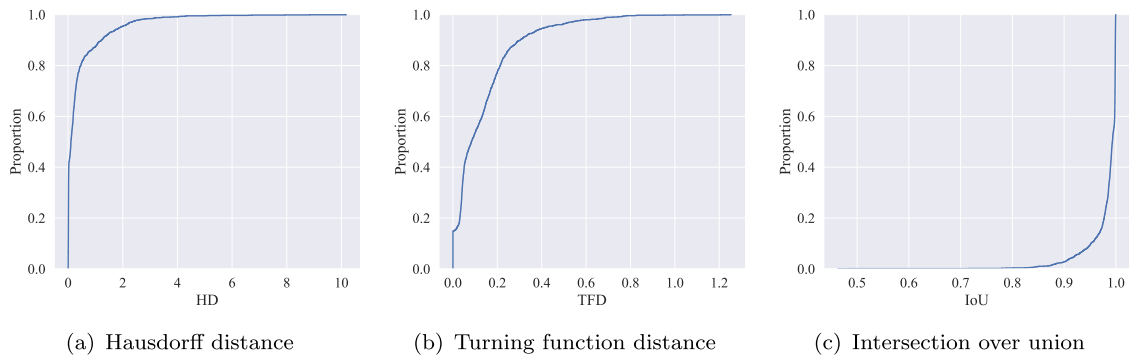


Fig. 9. Empirical cumulative distributions (eCD) of evaluation results with the three metrics.

Table 2

Evaluation results of the simplified buildings reconstructed from the predicted removal and movement labels.

Metrics	Max	Min	Mean	Median	Std
HD (m)	10.175	0.002	0.390	0.105	0.793
TFD	1.252	0.000	0.133	0.083	0.149
IoU (%)	1.000	0.463	0.983	0.993	0.033

Table 2 reports the key numerical values corresponding to the eCD curves in Fig. 9. While these values reflect that the proposed method generally performs well on the prediction of simplified buildings as depicted in Fig. 9, it should be noted that some extreme cases still show the limitations of the proposed method. For example, the maximum positional error has 10.175 m of HD and the maximum shape difference between the prediction and the ground truth is 1.252 of TFD.

In Fig. 10, for the cases from the 0th-quintile to 3rd-quintile of HD and TFD and from the 1st-quintile to 4th-quintile of 1-IoU,⁷ the building polygons generalized by our method generally fit well with the corresponding building of the ground truth target map. However, the buildings of the 3rd-quintile of TFD (sub-figure k) and of the 1st-quintile of 1-IoU (sub-figure l) have visible deviations (highlighted by red ellipses) from the ground truth buildings, representing the typical errors of our method. Fig. 10(l) shows movement regression errors that negatively impact the rectangularity of buildings, while the gap circled in Fig. 10(k) is due to misclassification. Besides, it is shown that the highest HD and TFD and the lowest IoU as reported in Table 2 all come from the same building, where several nodes in the corresponding building graph are misclassified.

4.3.2. Local evaluation of buildings at different simplification levels

In a realistic building simplification process carried out by cartographers, the geometric complexity of individual buildings determines how much detail should be simplified. It also influences how much knowledge the data-driven deep learning models should learn. Therefore, in this section, we calculated the IoU between the source map and the ground truth target map and categorized them into different simplification levels based on the step size of 0.05, which resulted in five levels ranging from 0.75 to 1.00.⁸ The lower the IoU between source and target is, the higher the simplification that needs to take place, and thus the more complex the learning task for the deep learning model.

Looking at the median values of these five levels of source-to-target IoU (Fig. 11), it is clear that the lower the level of simplification is, the higher the IoU that our method achieved. Despite there being only 14

buildings at the highest simplification level [0.75, 0.80), their prediction-to-target IoU values range from around 0.79 to 0.96 and show that MT_GCNN still manages rather well to learn to simplify these difficult cases. The extremely bad cases (prediction-to-target IoU < 0.75) of the whole test take place at the median simplification level [0.85, 0.90), suggesting that the simplification level may not be the determinant factor of the inferior limit of our proposed method.

Similar to the observation from Fig. 11, the simplified buildings in Fig. 12 at the lower simplification levels, e.g., [0.90, 0.95) and [0.95, 1.00], fit nearly 100% with the ground truth target buildings. By contrast, at higher simplification levels, from 0.75 to 0.90, there are more simplified buildings having obvious gaps with target buildings. For example, there are major missing parts for the predicted buildings in 11(r), (v), and (w) because our model removed nodes that should have been kept for the target scale. For these cases with a smaller difference like in 11(m), (p), and (u), the major reason is the misclassification between the two labels *kept static* and *being moved*. Furthermore, the buildings in Fig. 11(l) and (q) have achieved IoU values above 0.90 but their shapes are quite different from their target buildings due to the particularly large movement to predict.

4.4. Parameter setting analysis

To extract the practical knowledge of applying the deep learning method to the building simplification of vector maps, we conducted a series of parameter setting analyses. It should be clarified that the tested parameters refer to the generalized operational components in the method beyond the numerical setting of hyper-parameters in a deep learning model. Specifically, the analyzed parameters include the geometric features characterizing the polygon graph (Section 4.4.1), the building complexity involved in the dataset (Section 4.4.2), and the graph convolution operators in the proposed MT_GCNN model (Section 4.4.3).

4.4.1. Feature engineering

As introduced in Yan et al. (2021), a node in a polygon graph is characterized by its local structure (area, angle, and length) and regional structure (area, angle, perimeter, and radius) relative to the centroid of the polygon. For more detail, we refer to their article. In Fig. 13 these features are denoted by *loc_tri_area*, *loc_rot_angle*, *loc_rot_dir*, *chord_length*, *reg_tri_area*, *reg_angle*, *reg_semi_per*, and *reg_radius*. Besides, we also considered the length of the preceding edge and succeeding edge of the node, which is important information to simplify lines and polygons (Du et al., 2022).

We explored the importance of these commonly used features to building simplification by removing, in turn, individual features and observing the final loss change of the trained model compared to the training loss considering all features. The loss change is depicted in Fig. 13, where the loss increases when removing the rotation angle and direction, the length of the preceding edge, and the succeeding edge,

⁷ To achieve visual consistency over all results depicted in Fig. 10, we used the reverse order of IoU, i.e., 1-IoU as the visual evaluation metric.

⁸ There is only one building with an IoU below 0.75, more precisely 0.739, so we categorized it into the class [0.75, 0.80).

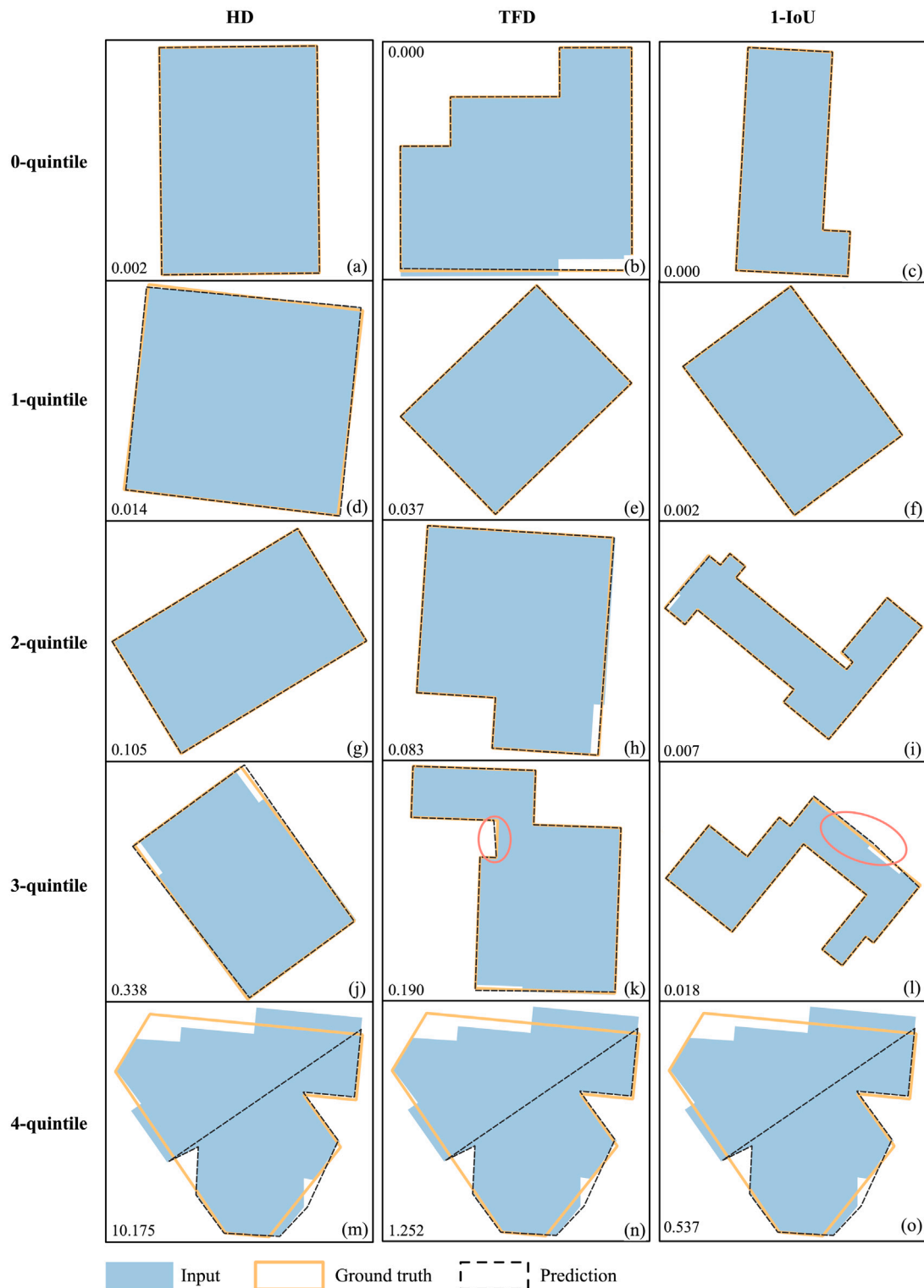


Fig. 10. Visual evaluation of the building simplification products at certain quintiles of HD, TFD, and IoU. In the sub-figures, the source map buildings are shaded in blue color, the corresponding simplified buildings of the ground truth target map are bounded by yellow lines, and the building simplification products resulting from our method are bounded by black dash lines. Besides, the numbers in the bottom left corners of each sub-figure correspond to the measured numerical value of the particular metric for the given test building. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

while the loss further decreases with the removal of other features. Such changes mean that the four increased features positively influence the performance of the model, which is also our rationale to solely consider the rotation angle and direction, and the length of the preceding edge and succeeding edge, as introduced in Section 3.1.

4.4.2. Building complexity control

As introduced in Section 4.1.1, our training set contains one half simplified buildings and one half non-simplified buildings. Theoretically, these non-simplified buildings commonly correspond to rectangular buildings and are easy to learn. Thus, we compared the model

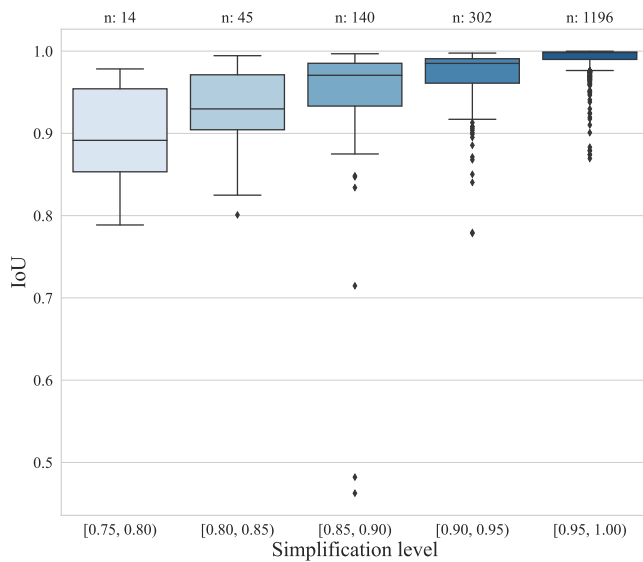


Fig. 11. The boxplot of the IoU of test buildings at different simplification levels. The values above each box denote the number of buildings contained in a particular class.

Table 3

Comparison between including non-simplified buildings and excluding non-simplified buildings.

Building complexity	HD	TFD	IoU	Failure cases
Including	0.083	0.105	0.993	0
Excluding	0.153	0.197	0.983	6

performance before and after excluding these easy cases (Table 3). It is shown that when the non-simplified buildings are excluded from the dataset, the HD and TFD for test buildings basically double and the IoU decreases markedly. Furthermore, six buildings failed to be reconstructed due to the misclassification of removal labels after excluding non-simplified buildings. These differences indicate that the non-simplified buildings should still be taken into account in balanced numbers as other simplified buildings for the training of a DL model, even though they are easily to be recognized as not being simplified.

4.4.3. Graph convolutional operation selection

The graph convolutional operation is the essential component of the proposed MT_GCNN model. Therefore, we compared the introduced GraphSAGE operation, which is the representative model of inductive learning, with two other classical transductive learning-based graph operations, GCN (Kipf and Welling, 2017) and GAT (Velickovic et al., 2018). The graph structures used for the three types of graph convolutional operations are the same. The results of this comparison are shown in Table 4. While the MT_GCNN model that uses GraphSAGE performed better than those adopting GCN and GAT operations in both node removal classification and node movement regression tasks, its computational time was also much shorter than the other two.

5. Discussion

5.1. Advantages of multi-task learning

The internal evaluation results in Section 4.2 suggest that multi-task learning can benefit the node movement regression while having no negative impact on the node removal classification compared with learning individual tasks. Hence, this result confirmed our intuition that classifying node removal before moving nodes would work better than directly moving nodes for simplifying buildings. Despite the slight improvement reflected by numerical values (MAE of preceding

movement: 0.428 $\rightarrow$ 0.390; MAE of succeeding movement: 0.424 $\rightarrow$ 0.388), each small improvement may significantly benefit the final products considering its large number of cases. Therefore, it is still highly recommended to introduce the multi-task learning paradigm to deep learning-based map generalization when multiple generalization operators are involved. Another rationale for the recommendation is that multi-task learning has sufficient flexibility to allow multiple cartographic constraints to be included in the map generalization (Hoarau, 2011; Balley et al., 2014), including position, orientation, topology, shape, and minimum dimensions (e.g. shortest edge length). In this research, the used negative log-likelihood loss for node removal classification and mean absolute error loss for node movement regression jointly contribute to the position constraint on individual buildings.

5.2. Feasibility of node removal classification and movement regression

Deep-learning approaches to the generalization of vector maps are rare so far. This research mitigates this gap by formulating the building simplification as a joint task of node removal classification and node movement regression that are learnable by deep learning models. The external evaluation with around 1700 test buildings in Section 4.3 showed that 80% of simplified buildings by our method hold positional errors below 0.5 m of HD, 95% with a shape difference under 0.5 of TFD, and around 98% with an area difference under 0.1 of IoU. These results demonstrate that it is feasible to use a vector map as a basis for building simplification, and they demonstrate that the proposed MT_GCNN model does a good job at this.

It is not new to decouple map generalization into node-level elementary operations such as removing nodes and moving nodes as introduced in this research. However, in existing algorithm- and rule-based generalization methods the trigger of specific operations commonly depends on the structure of neighboring edges of building polygons identified in a preceding structure recognition step, such as offset, extrusion, and corner (Sester, 2005). By contrast, our method is free from identifying these specific building structures because of the data-driven learning and non-linear activation function inherent to the proposed MT_GCNN model.

Recently, Yang et al. (2022) applied the ensemble mechanism to a classifier for automatically selecting the most appropriate one from a set of rule-based algorithms based on building shapes. Consequently, such a method needs a large set of features (about 30) and extensive feature engineering required for the set of multiple building simplification algorithms, while resulting in more reasonable simplification compared with individual algorithms. Conversely, our proposed method solely takes four features related to polygon graph nodes as input (see Section 4.4.1) and can achieve satisfying results, which shows the potential of using a deep learning approach for building polygons by node-level operations.

Compared to the image-based deep learning solution of Feng et al. (2019), our vector-based DL solution can result in errors (see Table 1) that are smaller than the grid size of 0.5 m used by Feng et al. (2019) to generate the image maps from the same Stuttgart dataset. Although this result should be interpreted with caution, as a different sample of buildings was used for the evaluation, it is a positive sign to indicate that our solution can reduce the positional errors of generalized building polygons.

5.3. Influence of input features and building complexity

As shown in Section 4.4, data is the key to training the proposed model besides the introduced graph convolutional operations. On the one hand, it is certainly not efficient for building simplification to input a wide range of node features that characterize building structures as introduced in Yan et al. (2021) and Du et al. (2022). According to Fig. 13, there are only four out of ten features that positively contribute to building simplification using GCNN and it is interesting to

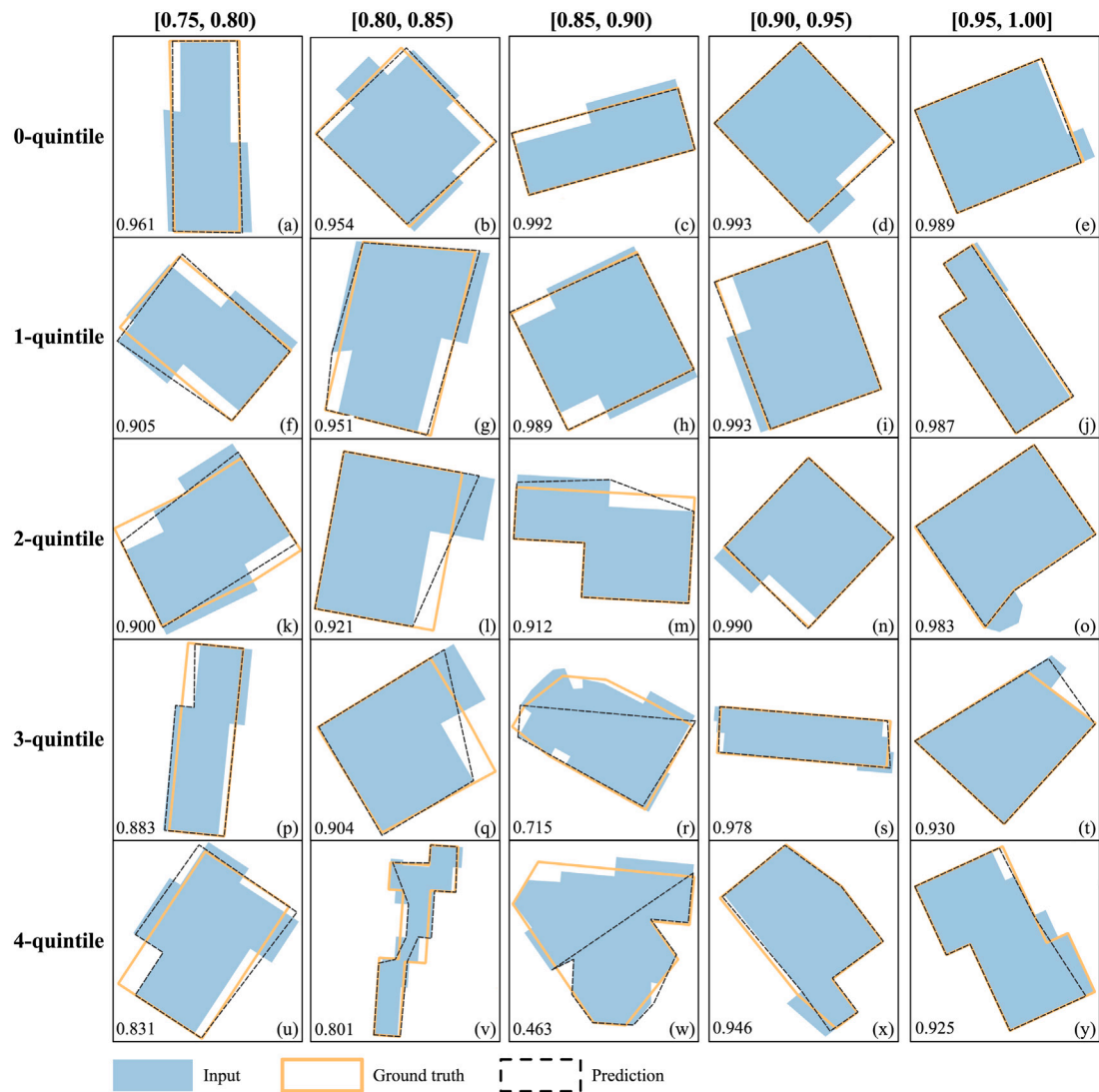


Fig. 12. Visual evaluation of the building simplification products at different simplification levels. The numbers in the bottom left corners of these sub-figures correspond to the prediction-to-target IoU. Each column presents the buildings at a certain simplification level.

Table 4

Comparison results of the GraphSAGE used in our method with other graph convolution operations. The best results are highlighted in bold.

Graph convolution operations	Node removal		Node movement – preceding	Node movement – succeeding	Computational time for 500 epochs (h)
	Accuracy	F1	MAE (m)	MAE (m)	
GCN	0.834	0.909	0.436	0.435	1.25
GAT	0.822	0.902	0.424	0.426	1.67
GraphSAGE	0.862	0.926	0.390	0.388	0.69

see that these useful features all belong to the local ones of a building polygon (Yan et al., 2021), suggesting that the building simplification mainly focuses on the local context of building footprints (Sester and Brenner, 2005; Mackaness et al., 2014). Thus, it may be reasonable for data-driven map generalization models (Du et al., 2022; Yang et al., 2022) to extract important features to reduce the model complexity, rather than working with a broad range of unspecific features. While such results are not surprising because our dataset is processed by the CHANGE software that utilizes rule-based algorithms to simplify buildings, they in turn reflect that our method does effectively learn the cartographic knowledge for building simplification.

On the other hand, it should be noted that the building complexity involved in the dataset should also be carefully considered. In

Section 4.4.2, we compared the performance of the model between including and excluding non-simplified buildings. The results demonstrate that the building complexity does impact the generality of the trained model. Meanwhile, the local evaluations in Section 4.3.2 also underline such a significant influence. However, the shape complexity of building polygons can be measured from multiple aspects, such as the vertex number of footprint, rectangularity, etc. Zhang and Lu (2004), Fan et al. (2014) and Basaraner and Cetinkaya (2017). It is worth empirically investigating the impact of the training datasets based on relevant aspects (sample size, balanced distribution of buildings by shape complexity, etc.) on the generality of the derived building simplification models.

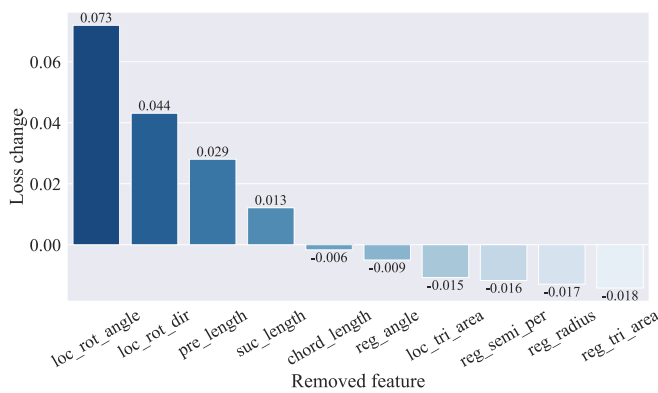


Fig. 13. Loss change of the proposed DL model after removing a particular feature, compared with a model that takes all features.

5.4. Limitations and future work

While several experiments demonstrated that the proposed method can successfully deal with the majority of building simplification cases, there are a few extreme cases contributing to the lower performance bound of the proposed method. As detailed in Fig. 12, these cases are mainly those buildings with irregular shapes that pose much more difficulty to the model to learn the appropriate cartographic knowledge. Meanwhile, the small number of such irregular buildings also impedes the model from effectively learning to simplify these buildings. Thus, it is important to build a dataset containing an equivalent number of buildings with different shapes (Yan et al., 2022) to improve the generality of the trained MT_GCNN model.

Theoretically, our trained models can be applied to other map generalization tasks by being fine-tuned with other independent data sources. However, the capacity of our trained models relies on scale changes and geographic contexts. Because the usage of map generalization operators is scale-dependent (Cecconi et al., 2002), creating multiple datasets is required for training models for generalization at varying scales. Notably, each trained model may apply to a certain scale change since the generalization criteria for different scale changes are quite different (Regnauld and McMaster, 2007).

Furthermore, the geographic contexts also matter in the transferability of such a trained model for building simplification. For instance, a model trained in an urban area can work better in a similar urban area than in a mountain area. Currently, various geographic embedding methods (Fu et al., 2021; Wang and Biljecki, 2022) have the potential to quantify the difference between geographic contexts and thus to evaluate the fine-tuning efforts for transferring a model to different geographic regions. Therefore, it will be challenging but also valuable to explore how the trained models for different geographic regions and scale changes can be transferred.

Regarding the graph definition, while it is intuitive and straightforward to enable the graph construction, the currently used approach may not support the representation of complex building polygons including ones with holes. Theoretically, a triangulated graph structure should have a better capacity to uniformly represent polygons no matter how complex they are (Mai et al., 2022b). However, such a structure will remove the sequential order of vertices along polygonal boundaries, requiring a model capable of reconstructing the features characterizing the building polygons. Therefore, the influence of a graph definition in GCNN-based building simplification will have to be further investigated in the future.

Considering that the node correspondence between source and target buildings is certainly not unique, there might be some alternative solutions to label node removals and movements, which may also influence the model performance. It is thus worth applying the

encoder-decoder architecture (Yu and Chen, 2022) to identify the transformation at the graph level beyond the node level as introduced in this research.

Additionally, this research mainly focuses on the simplification operators for individual buildings but neglects other operators across multiple buildings, such as amalgamation and typification. Given the advantage of image-based deep learning methods (Feng et al., 2019; Kang et al., 2019) on unifying map objects as grids, it will be interesting to integrate our vector-based solution and these image-based solutions to produce ultimate map generalization solutions at multiple scales.

6. Conclusions

Identifying the lack of deep learning solutions for building simplification using vector maps, and positing that vector-based solutions would further benefit the automation of map generalization, we proposed a multi-task learning method with graph convolutional neural networks. The polygonal boundaries of buildings are modeled as polygon graphs, and the building simplification problem is formulated as a joint task of node removal classification and node movement regression in polygon graphs. A multi-task graph convolutional neural network, MT_GCNN, has been developed to simultaneously learn node removal and movement knowledge. We conducted a series of experiments to comprehensively evaluate the performance of the proposed method with around 1700 test buildings. The evaluation results show that the proposed method is capable of generating reasonable simplification results for most test buildings with low HD and TFD, and high IoU, demonstrating the promising capability of our method for building simplification.

In the future, as irregular shapes of buildings may lead to poor results, we will have further improvements of the method by using more balanced training data that contain more buildings with a greater diversity of shapes, and by exploring other learning strategies. Meanwhile, we plan to apply other graph structures and end-to-end DL architectures to explore the capacity of DL models for the building simplification task. Further, we will also try to integrate image-based DL solutions and vector-based DL solutions for producing mature generalized maps.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research was supported by the Swiss National Science Foundation through project number 200021_204081 DeepGeneralization.

References

- Ai, T., Li, Z., Liu, Y., 2005. Progressive transmission of vector data based on changes accumulation model. In: *Developments in Spatial Data Handling*. Springer, pp. 85–96.
- Arkin, E.M., Chew, L.P., Huttenlocher, D.P., Kedem, K., Mitchell, J.S., 1991. An efficiently computable metric for comparing polygonal shapes. *IEEE Trans. Pattern Anal. Mach. Intell.* 13 (3), 209.
- Bailey, S., Baella, B., Christophe, S., Pla, M., Regnauld, N., Stoter, J., 2014. Map specifications and user requirements. In: Burghardt, D., Duchêne, C., Mackaness, W. (Eds.), *Abstracting Geographic Information in a Data Rich World*. Springer International Publishing, pp. 17–52.
- Barraut, M., Regnauld, N., Duchêne, C., Haire, K., Baeijs, C., Demazeau, Y., Hardy, P., Mackaness, W., Ruas, A., Weibel, R., 2001. Integrating multi-agent, object-oriented and algorithmic techniques for improved automated map generalization. In: *Proceedings 20th International Cartographic Conference*, no. 1. pp. 2110–2116.
- Basaraner, M., Cetinkaya, S., 2017. Performance of shape indices and classification schemes for characterising perceptual shape complexity of building footprints in GIS. *Int. J. Geogr. Inf. Sci.* 31 (10), 1952–1977.

- Bayer, T., 2009. Automated building simplification using a recursive approach. In: *Cartography in Central and Eastern Europe*. Springer, pp. 121–146.
- Bildirci, I.O., 2004. Building and road generalization with the Change generalization software using turkish topographic base map data. *Cartogr. Geogr. Inf. Sci.* 31 (1), 43–54.
- Brassel, K.E., Weibel, R., 1988. A review and conceptual framework of automated map generalization. *Int. J. Geogr. Inf. Syst.* 2 (3), 229–244.
- Buchin, K., Meulemans, W., Renssen, A.V., Speckmann, B., 2016. Area-preserving simplification and schematization of polygonal subdivisions. *ACM Trans. Spat. Algorithms Syst. (TSAS)* 2 (1), 1–36.
- Cecconi, A., Weibel, R., Barrault, M., 2002. Improving automated generalisation on on-demand web mapping by multiscale databases. In: *Advances in Spatial Data Handling: 10th International Symposium on Spatial Data Handling*. Springer, pp. 515–531.
- Cheng, B., Liu, Q., Li, X., Wang, Y., 2013. Building simplification using backpropagation neural networks: a combination of cartographers' expertise and raster-based local perception. *GISci. Remote Sens.* 50 (5), 527–542.
- Courtial, A., Touya, G., Zhang, X., 2023. Deriving map images of generalised mountain roads with generative adversarial networks. *Int. J. Geogr. Inf. Sci.* 37 (3), 499–528.
- Douglas, D.H., Peucker, T.K., 1973. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartogr.: Int. J. Geogr. Inf. Geovis.* 10 (2), 112–122.
- Du, J., Wu, F., Yin, J., Liu, C., Gong, X., 2022. Polyline simplification based on the artificial neural network with constraints of generalization knowledge. *Cartogr. Geogr. Inf. Sci.* 49 (4), 313–337.
- Fan, H., Zipf, A., Fu, Q., Neis, P., 2014. Quality assessment for building footprints data on OpenStreetMap. *Int. J. Geogr. Inf. Sci.* 28 (4), 700–719.
- Feng, Y., Thiemann, F., Sester, M., 2019. Learning cartographic building generalization with deep convolutional neural networks. *ISPRS Int. J. Geo-Inf.* 8 (6), 258.
- Fu, C., Nanni, M., Yeghikyan, G., Weibel, R., 2021. Measuring polycentricity: a whole graph embedding perspective. In: *GIScience 2021 Short Paper Proceedings*. online.
- Hamilton, W.L., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. In: Guyon, I., von Luxburg, U., Bengio, S., Wallach, H.M., Fergus, R., Vishwanathan, S.V.N., Garnett, R. (Eds.), *Proceedings of the 31st International Conference on Neural Information Processing Systems*. pp. 1024–1034.
- Harrie, L., Weibel, R., 2007. Modelling the overall process of generalisation. In: *Generalisation of Geographic Information*. Elsevier, pp. 67–87.
- Hauert, J.-H., 2012. A symmetry detector for map generalization and urban-space analysis. *ISPRS J. Photogramm. Remote Sens.* 74, 66–77.
- Hauert, J.-H., Wolff, A., 2010. Optimal and topologically safe simplification of building footprints. In: *Proceedings of the 18th Sigspatial International Conference on Advances in Geographic Information Systems*. pp. 192–201.
- He, X., Zhang, X., Xin, Q., 2018. Recognition of building group patterns in topographic maps based on graph partitioning and random forest. *ISPRS J. Photogramm. Remote Sens.* 136, 26–40.
- Hoarau, C., 2011. Reaching a compromise between contextual constraints and cartographic rules: Application to sustainable maps. *Cartogr. Geogr. Inf. Sci.* 38 (2), 79–88.
- Kang, Y., Gao, S., Roth, R.E., 2019. Transferring multiscale map styles using generative adversarial networks. *Int. J. Cartogr.* 5 (2–3), 115–141.
- Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization. In: Bengio, Y., LeCun, Y. (Eds.), *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings*.
- Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks. In: *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*.
- Liebel, L., Körner, M., 2018. Auxiliary tasks in multi-task learning. *Computing Research Repository*, abs/1805.06334.
- Liu, Y., Zhang, J., Fang, L., Jiang, Q., Zhou, B., 2021. Multimodal motion prediction with stacked transformers. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR*, pp. 7577–7586.
- Ma, D., Zhao, Z., Zheng, Y., Guo, R., Zhu, W., 2020. PolySimp: A tool for polygon simplification based on the underlying scaling hierarchy. *ISPRS Int. J. Geo-Inf.* 9 (10), 594.
- Mackaness, W., Burghardt, D., Duchêne, C., 2014. Map generalisation: Fundamental to the modelling and understanding of geographic space. In: *Abstracting Geographic Information in a Data Rich World*. Springer, pp. 1–15.
- Mai, G., Janowicz, K., Hu, Y., Gao, S., Yan, B., Zhu, R., Cai, L., Lao, N., 2022a. A review of location encoding for GeoAI: methods and applications. *Int. J. Geogr. Inf. Sci.* 36 (4), 639–673.
- Mai, G., Jiang, C., Sun, W., Zhu, R., Xuan, Y., Cai, L., Janowicz, K., Ermon, S., Lao, N., 2022b. Towards general-purpose representation learning of polygonal geometries. *GeoInformatica* 1–52.
- Powitz, B., 1993. Computer-assisted generalization-an important software tool in GIS. *Int. Arch. Photogramm. Remote Sens.* 29, 664.
- Regnauld, N., Edwards, A., Barrault, M., 1999. Strategies in building generalisation: modelling the sequence, constraining the choice. In: *ICA Workshop on Progress in Automated Map Generalization*, Ottawa, Canada. pp. 372–378.
- Regnauld, N., McMaster, R.B., 2007. A synoptic view of generalisation operators. In: *Generalisation of Geographic Information*. Elsevier, pp. 37–66.
- Ruas, A., Duchêne, C., 2007. Chapter 14 - A prototype generalisation system based on the multi-agent system paradigm. In: Mackaness, W.A., Ruas, A., Sarjakoski, L.T. (Eds.), *Generalisation of Geographic Information*. In: *International Cartographic Association*, Elsevier Science B.V., Amsterdam, pp. 269–284.
- Samsonov, T.E., Yakimova, O.P., 2017. Shape-adaptive geometric simplification of heterogeneous line datasets. *Int. J. Geogr. Inf. Sci.* 31 (8), 1485–1520.
- Sester, M., 2005. Optimization approaches for generalization and data abstraction. *Int. J. Geogr. Inf. Sci.* 19 (8–9), 871–897.
- Sester, M., Brenner, C., 2005. Continuous generalization for visualization on small mobile devices. In: *Developments in Spatial Data Handling*. Springer, pp. 355–368.
- Shen, Y., Ai, T., Li, C., 2019. A simplification of urban buildings to preserve geometric properties using superpixel segmentation. *Int. J. Appl. Earth Obs. Geoinf.* 79, 162–174.
- Touya, G., Zhang, X., Lokhat, I., 2019. Is deep learning the new agent for map generalization? *Int. J. Cartogr.* 5 (2–3), 142–157.
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks. In: *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net.
- Wang, L., Ai, T., Burghardt, D., Shen, Y., Yang, M., 2023. A hexagon-based method for polygon generalization using morphological operators. *Int. J. Geogr. Inf. Sci.* 37 (1), 88–117.
- Wang, J., Biljecki, F., 2022. Unsupervised machine learning in urban studies: A systematic review of applications. *Cities* 129, 103925.
- Wang, H.-l., Wu, F., Zhang, L., Deng, H., 2005. The application of mathematical morphology and pattern recognition to building polygon simplification. *Acta Geod. Cartogr. Sin.* 34 (3), 269–276.
- Weibel, R., Keller, S., Reichenbacher, T., 1995. Overcoming the knowledge acquisition bottleneck in map generalization: The role of interactive systems and computational intelligence. In: Frank, A.U., Kuhn, W. (Eds.), *Spatial Information Theory a Theoretical Basis for GIS*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 139–156.
- Xiao, T., Ai, T., Yu, H., Yang, M., Liu, P., 2023. A point selection method in map generalization using graph convolutional network model. *Cartogr. Geogr. Inf. Sci.* 1–21.
- Yan, X., Ai, T., Yang, M., Tong, X., 2021. Graph convolutional autoencoder model for the shape coding and cognition of buildings in maps. *Int. J. Geogr. Inf. Sci.* 35 (3), 490–512.
- Yan, X., Ai, T., Yang, M., Tong, X., Liu, Q., 2022. A graph deep learning approach for urban building grouping. *Geocarto Int.* 37 (10), 2944–2966.
- Yan, X., Ai, T., Yang, M., Yin, H., 2019. A graph convolutional neural network for classification of building patterns using spatial vector data. *ISPRS J. Photogramm. Remote Sens.* 150, 259–273.
- Yan, X., Ai, T., Zhang, X., 2017. Template matching and simplification method for building features based on shape cognition. *ISPRS Int. J. Geo-Inf.* 6 (8), 250.
- Yang, M., Yuan, T., Yan, X., Ai, T., Jiang, C., 2022. A hybrid approach to building simplification with an evaluator from a backpropagation neural network. *Int. J. Geogr. Inf. Sci.* 36 (2), 280–309.
- Yu, W., Chen, Y., 2022. Data-driven polyline simplification using a stacked autoencoder-based deep neural network. *Trans. GIS* 26 (5), 2302–2325.
- Zhang, D., Lu, G., 2004. Review of shape representation and description techniques. *Pattern Recognit.* 37 (1), 1–19.
- Zhou, Z., Fu, C., Weibel, R., 2022. Building simplification of vector maps using graph convolutional neural networks. In: *Abstracts of the ICA*, Vol. 5. p. 86.